LLMs with PyTorch

CHEAT SHEET

1 Tokenize › 2 Build or load a model › 3 Pretrain or finetune › 4 Generate & evaluate

General

A GPT-style LLM is a **decoder-only Transformer** trained on **next-token prediction**: given tokens $t_1 \dots t_n$, predict t_{n+1}. The loss is **cross-entropy** between the logits and the inputs **shifted by one**. Finetuning reuses the same loss on curated data.

B	batch size	**V**	vocabulary size
T	sequence / context length	**H**	attention heads
d	model (embedding) dim	**L**	Transformer blocks

*Params ≈ **12·L·d² + V·d**. Shapes flow as **[B, T]** → **[B, T, d]** → **[B, T, V]**.*

Building blocks

nn.Embedding(V, d): token id → vector (+ position)

F.scaled_dot_product_attention(q,k,v, **is_causal=True**): fused causal attention

nn.RMSNorm(d): cheaper LayerNorm, used **pre**-attention & pre-MLP

MLP / SwiGLU: d → 4d → d, applied to each token

RoPE: rotary position encoding on q & k (Llama, Qwen)

Residual x + f(x) around every sub-layer; **weight tying** head ↔ embedding

Tokenize

Text → sub-word ids with **BPE**. Always use the tokenizer and **chat template** that match the model.

```python
from transformers import AutoTokenizer
tok = AutoTokenizer.from_pretrained("Qwen/Qwen3-0.6B")
ids = tok("Hello world", return_tensors="pt").input_ids  # [1,T]
msgs = [{"role": "user", "content": "Explain LoRA"}]
prompt = tok.apply_chat_template(msgs, tokenize=False,
                                 add_generation_prompt=True)
tok.pad_token = tok.pad_token or tok.eos_token  # for batching
```

SMALL OPEN MODELS (HUGGING FACE IDS)

HuggingFaceTB/SmolLM2-135M	135M / 360M / 1.7B, great for laptops
Qwen/Qwen3-0.6B	0.6B / 1.7B / 4B, strong multilingual
google/gemma-3-270m	270M / 1B, built for task finetunes
meta-llama/Llama-3.2-1B	1B / 3B, gated (accept license)

Generate

```python
inputs = tok(prompt, return_tensors="pt").to(dev)
out = model.generate(**inputs, max_new_tokens=200,
                     do_sample=True, temperature=0.7, top_p=0.9)
print(tok.decode(out[0], skip_special_tokens=True))
```

greedy / temperature	argmax, or logits / τ (<1 sharper, >1 more random)
top_k / top_p	sample only from the k best / p-mass tokens
KV cache	reuse past keys/values so each step is O(T)

Build a tiny GPT

```python
import torch, torch.nn as nn, torch.nn.functional as F

class Block(nn.Module):
    def __init__(self, d, h):
        super().__init__()
        self.h, self.n1, self.n2 = h, nn.RMSNorm(d), nn.RMSNorm(d)
        self.qkv = nn.Linear(d, 3 * d, bias=False)
        self.proj = nn.Linear(d, d, bias=False)
        self.mlp = nn.Sequential(nn.Linear(d, 4 * d), nn.GELU(),
                                 nn.Linear(4 * d, d))
    def forward(self, x):                       # x: [B, T, d]
        B, T, C = x.shape
        q, k, v = self.qkv(self.n1(x)).split(C, dim=-1)
        q, k, v = (t.view(B, T, self.h, -1).transpose(1, 2)
                   for t in (q, k, v))          # [B, h, T, d/h]
        a = F.scaled_dot_product_attention(q, k, v, is_causal=True)
        x = x + self.proj(a.transpose(1, 2).reshape(B, T, C))
        return x + self.mlp(self.n2(x))
class TinyGPT(nn.Module):
    def __init__(self, V, d=384, h=6, L=6, T=256):
        super().__init__()
        self.tok, self.pos = nn.Embedding(V, d), nn.Embedding(T, d)
        self.blocks = nn.Sequential(*(Block(d, h) for _ in range(L)))
        self.norm = nn.RMSNorm(d)
        self.head = nn.Linear(d, V, bias=False)
        self.head.weight = self.tok.weight      # weight tying
    def forward(self, idx, targets=None):       # idx: [B, T]
        pos = torch.arange(idx.size(1), device=idx.device)
        x = self.blocks(self.tok(idx) + self.pos(pos))
        logits = self.head(self.norm(x))        # [B, T, V]
        if targets is None: return logits, None
        loss = F.cross_entropy(logits.flatten(0, 1),
                               targets.flatten(), ignore_index=-100)
        return logits, loss
```

Pretraining loop

```python
data = torch.tensor(tok.encode(text))       # 1-D token stream
def get_batch(B=32, T=256):
    i = torch.randint(len(data) - T - 1, (B,))
    x = torch.stack([data[j:j + T] for j in i])
    y = torch.stack([data[j + 1:j + T + 1] for j in i])
    return x.to(dev), y.to(dev)
model = torch.compile(TinyGPT(V=len(tok)).to(dev))
opt = torch.optim.AdamW(model.parameters(), lr=3-4,
                        betas=(0.9, 0.95), weight_decay=0.1)
for step in range(5000):
    x, y = get_batch()
    with torch.autocast("cuda", dtype=torch.bfloat16):
        _, loss = model(x, y)
    loss.backward()
    nn.utils.clip_grad_norm_(model.parameters(), 1.0)
    opt.step(); sched.step(); opt.zero_grad(set_to_none=True)
```

***y** is **x** shifted by one token. Use **warmup + cosine decay** for **sched**. Track **perplexity = exp(val loss)**; random-init loss ≈ **ln V**.*

EFFICIENCY & MEMORY TRICKS

dtype=torch.bfloat16	Half the memory, no loss scaling (unlike fp16)
model.gradient_checkpointing_enable()	Less activation memory, ~30% slower
attn_implementation="sdpa"	Fused attention (or "flash_attention_2")
gradient_accumulation_steps=8	Large effective batch on a small GPU

Finetuning

Full finetune	Update every weight: best quality, most memory
LoRA	Freeze W, learn low-rank ΔW = B·A (~0.1–1% params)
QLoRA	LoRA on a frozen **4-bit NF4** base model
DoRA	Magnitude + direction LoRA (**use_dora=True**)
SFT → DPO	Instruction-tune, then align on preference pairs

GPU MEMORY PER PARAMETER (TRAINING)

Full FT, AdamW	~16 B: weights, grads, fp32 copy, Adam m & v
LoRA, bf16 base	~2 B + adapters + activations
QLoRA, 4-bit base	~0.5–0.7 B: a 7B model fits in ~6 GB

LoRA

h = W·x + (α/r)·B·A·x, A: r×k, B: d×r, r ≪ d. **B starts at 0**, so step 0 equals the base model.

```python
class LoRALinear(nn.Module):
    def __init__(self, base, r=8, alpha=16):
        super().__init__()
        self.base = base.requires_grad_(False)   # freeze W
        self.A = nn.Linear(base.in_features, r, bias=False)
        self.B = nn.Linear(r, base.out_features, bias=False)
        nn.init.zeros_(self.B.weight)            # dW = 0 at start
        self.s = alpha / r
    def forward(self, x):
        return self.base(x) + self.s * self.B(self.A(x))
from peft import LoraConfig, get_peft_model   # with PEFT
cfg = LoraConfig(r=16, lora_alpha=32, lora_dropout=0.05,
        target_modules="all-linear", task_type="CAUSAL_LM")
model = get_peft_model(model, cfg)
model.print_trainable_parameters()
```

*Typical: **r 8–64**, **α = r or 2r**, **lr 1e-4–2e-4** (≈10× full-FT lr).*

QLoRA

```python
from transformers import AutoModelForCausalLM
from transformers import BitsAndBytesConfig as BnB
from peft import prepare_model_for_kbit_training
bnb = BnB(load_in_4bit=True,
    bnb_4bit_quant_type="nf4",          # NormalFloat4
    bnb_4bit_use_double_quant=True,     # quantize scales too
    bnb_4bit_compute_dtype=torch.bfloat16)
model = AutoModelForCausalLM.from_pretrained("Qwen/Qwen3-0.6B",
    quantization_config=bnb, device_map="auto")
model = prepare_model_for_kbit_training(model)
model = get_peft_model(model, cfg)      # same LoRA config
```

SFT, SAVE & MERGE

labels = -100	Mask prompt tokens: loss on answers only
trl.SFTTrainer	Chat data + **peft_config**, packing, bf16
model.save_pretrained()	Saves the adapter only (a few MB)
model.merge_and_unload()	Fold BA into W (merge on a bf16 base)

github.com/arnabdeypolimi/pytorch-cheatsheets / CC BY 4.0 / v1.0 · 10/2026